Project Design Document

Milestone 3

Java Desktop Application using Swing UI and JDBC

Document Detail	Information
Project Title	[Enter project title]
Team Number	[Enter team number]
Class and Semester	[Enter class and semester]
Course	Java Programming
Milestone Marks	3 Marks
Submission Date	[DD Month YYYY]

Submitted by

[Team Lead and members]

This document converts the approved project requirements into an implementable design. Each diagram, class, table, screen and design decision must be traceable to one or more functional requirements.

Document control and team details

Version	Date	Prepared by	Changes	Status
0.1	[Date]	[Name]	Initial design submission	Draft
[ ]	[Date]	[Name]	[Summary of revision]	[Draft / Approved]

Team and HackerRank Java progress

Mandatory: Include this section in every project submission and PPT. Enter the Java stars visible on the review date and preserve the previous milestone value so that continuous practice can be monitored.

No.	Student	Roll no.	Role	HackerRank profile	M2 stars	Current stars	Change
1	[Name]	[Roll no.]	Team Lead	[Profile / username]	[0-5]	[0-5]	[+ / -]
2	[Name]	[Roll no.]	Member	[Profile / username]	[0-5]	[0-5]	[+ / -]
3	[Name]	[Roll no.]	Member	[Profile / username]	[0-5]	[0-5]	[+ / -]
4	[Name]	[Roll no.]	Member	[Profile / username]	[0-5]	[0-5]	[+ / -]
5	[Name]	[Roll no.]	Member	[Profile / username]	[0-5]	[0-5]	[+ / -]

Evidence: Add a working profile link or include a current profile screenshot in Appendix A.

Submission instructions

- Replace all bracketed prompts with project-specific content.
- Use consistent names and requirement IDs across the Requirements Document, Design Document and PPT.
- Diagrams must be readable, labelled and consistent with the written design.
- Do not include implementation code in this milestone unless a short signature or interface is needed to explain the design.

1 Design basis

Briefly state the approved project purpose and the design approach selected to implement it. Mention any requirements that changed after Milestone 2.

1.1 Requirements changes

Change ID	Requirement affected	Change and reason	Approved by
DC-01	[FR-__]	[No change / describe change and justification]	[Name / date]
DC-02	[FR-__]	[Describe another change, if any]	[Name / date]

1.2 System architecture

Use a layered desktop architecture. Show responsibilities and dependencies among the Swing user interface, application or service layer, domain classes, data access layer, JDBC and database.

Figure 1 System architecture diagram

[Insert the layered architecture diagram and show permitted dependencies]

[Diagram title / version / date]

Layer or component	Responsibility	Key classes or packages	Depends on
Presentation	Swing screens, events and user messages	[package / classes]	Application layer
Application / Service	Coordinates use cases and validations	[package / classes]	Domain and DAO
Domain	Represents business entities and rules	[package / classes]	[None / utility]
Data Access	Executes JDBC operations and maps records	[DAO classes]	JDBC and database

2 Use case design

Identify the users or actors and the system behaviour visible to them. Every major functional requirement should appear in at least one use case.

Figure 2 Use case diagram

[Insert the use case diagram with actors, system boundary and named use cases]

[Diagram title / version / date]

2.1 Actor responsibilities

Actor	Description	Main use cases	Related requirements
[Actor 1]	[Who the actor is]	[Use cases]	[FR-__]
[Actor 2]	[Who the actor is]	[Use cases]	[FR-__]
[Actor 3]	[Who the actor is]	[Use cases]	[FR-__]

2.2 Detailed use case specification

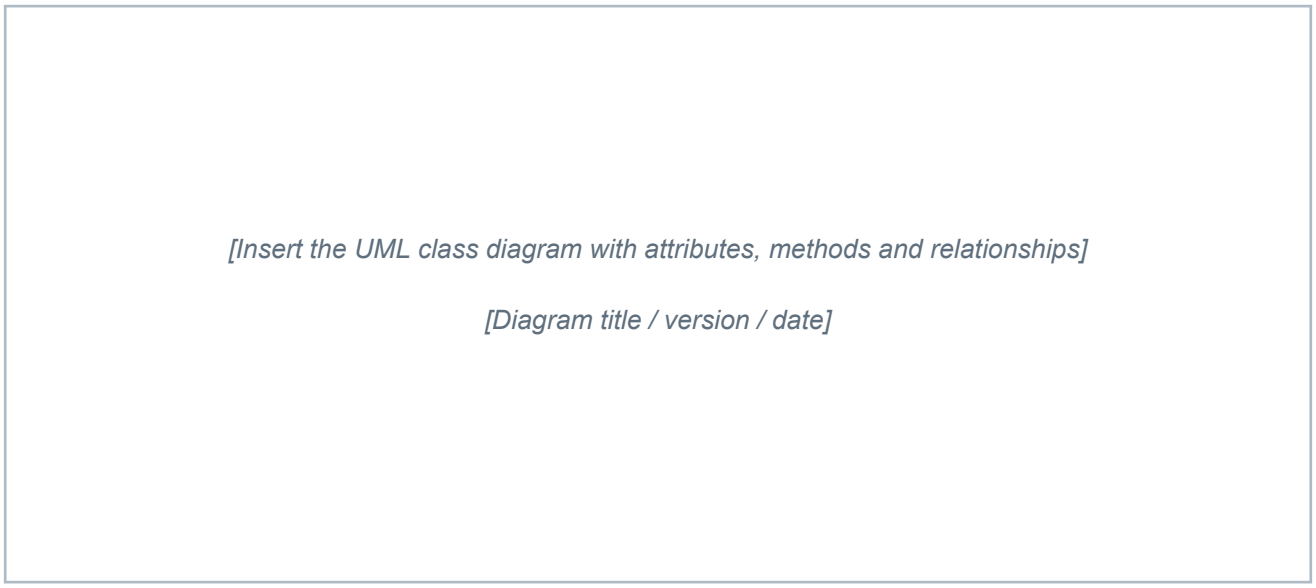
Field	Details
Use case ID and name	UC-01 [Use case name]
Primary actor	[Actor]
Preconditions	[What must already be true]
Trigger	[Event that starts the use case]
Main flow	1. [Step] 2. [Step] 3. [Step]
Alternate or error flow	[Validation failure / database error / cancellation]
Postconditions	[Successful result and stored data]
Related requirements	[FR-__ , FR-__]

Repeat the use case specification for each important user goal.

3 Class and object design

The class design must demonstrate appropriate object-oriented decomposition. Include class names, essential attributes, major methods, relationships and multiplicities. Avoid placing every operation in a single class.

Figure 3 UML class diagram



3.1 Class responsibilities

Class	Type	Responsibility	Key attributes and methods	Related FRs
[ClassName]	Entity / UI / Service / DAO / Utility	[Single clear responsibility]	[Key members]	[FR-__]
[ClassName]	[Type]	[Responsibility]	[Key members]	[FR-__]
[ClassName]	[Type]	[Responsibility]	[Key members]	[FR-__]
[ClassName]	[Type]	[Responsibility]	[Key members]	[FR-__]

3.2 Object-oriented design choices

- Encapsulation: [How object state is protected and updated]
- Inheritance or interfaces: [Where used and why; avoid unnecessary inheritance]
- Polymorphism: [Behaviour that varies by implementation, if applicable]
- Composition: [Objects owned or used by other objects]

4 Module and package design

Group classes according to responsibility and show which modules or packages collaborate. Package names should follow Java naming conventions.

Figure 4 Package or component diagram

[Insert the package or component diagram and show dependencies]

[Diagram title / version / date]

Module or package	Purpose	Main classes	Public services	Related FRs
[ui]	Swing screens and event handling	[Classes]	[Operations exposed]	[FR-__]
[service]	Application workflow and business rules	[Classes]	[Operations exposed]	[FR-__]
[model]	Domain data and behaviour	[Classes]	[Operations exposed]	[FR-__]
[dao]	JDBC persistence operations	[Classes]	[CRUD methods]	[FR-__]
[util / config]	Shared configuration and utilities	[Classes]	[Operations exposed]	[FR-__]

4.1 Proposed source structure

Project root: [project-name]

src/[base.package]/ui | service | model | dao | util

resources/ [configuration, icons or SQL files] docs/ [JavaDocs, user guide and test reports]

4.2 Design conventions

- Classes use PascalCase; methods and variables use camelCase; constants use UPPER_SNAKE_CASE.
- UI classes do not contain SQL; DAO classes do not control Swing components.
- Database connections, prepared statements and result sets are closed safely.
- Public classes and methods will receive meaningful JavaDoc comments before Code Review 2.

5 Database design

Translate the approved data requirements into entities, keys, relationships and constraints. The schema must support the functional requirements without unnecessary duplication.

Figure 5 Entity relationship diagram

[Insert the ER diagram with primary keys, foreign keys and cardinalities]

[Diagram title / version / date]

5.1 Table catalogue

Table	Purpose	Primary key	Foreign keys	Related FRs
[table_name]	[Stored information]	[column]	[column -> table]	[FR-__]
[table_name]	[Stored information]	[column]	[column -> table]	[FR-__]
[table_name]	[Stored information]	[column]	[column -> table]	[FR-__]
[table_name]	[Stored information]	[column]	[column -> table]	[FR-__]

5.2 Detailed table specification

Table name: [table_name] Purpose: [purpose]

Column	Data type	PK	FK	Null	Default	Validation or constraint
[column]	[type(size)]	Y/N	Y/N	Y/N	[value]	[Unique / range / format]
[column]	[type(size)]	Y/N	Y/N	Y/N	[value]	[Constraint]
[column]	[type(size)]	Y/N	Y/N	Y/N	[value]	[Constraint]
[column]	[type(size)]	Y/N	Y/N	Y/N	[value]	[Constraint]

Repeat the detailed table specification for every database table.

6 Swing user interface design

Show how users navigate between screens and how each screen supports a use case. Wireframes may be created using a GUI designer, drawing tool or hand-drawn sketch that is clear and labelled.

Figure 6 Screen navigation map

[Insert the navigation diagram from login or home screen to major screens]

[Diagram title / version / date]

6.1 Screen specification

Screen ID and name	User and purpose	Main components	Actions	Related FRs
UI-01 [Name]	[Actor and task]	[Fields, buttons, JTable, etc.]	[Save / search / update]	[FR-__]
UI-02 [Name]	[Actor and task]	[Components]	[Actions]	[FR-__]
UI-03 [Name]	[Actor and task]	[Components]	[Actions]	[FR-__]
UI-04 [Name]	[Actor and task]	[Components]	[Actions]	[FR-__]

Figure 7 Key Swing screen wireframe

[Insert one representative screen wireframe with labelled components]

[Diagram title / version / date]

6.2 User feedback and states

- Validation messages: [Where and how messages appear]
- Confirmation messages: [Actions requiring confirmation]
- Empty and loading states: [Expected user feedback]
- Keyboard, focus and tab order: [Basic usability approach]

7 Interaction and sequence design

Create sequence diagrams for at least two important workflows. Show the actor, Swing screen, service or controller, DAO, JDBC connection and database interaction.

Figure 8 Sequence diagram for [UC-01]

[Insert the sequence diagram for one create or transaction workflow]

[Diagram title / version / date]

Figure 9 Sequence diagram for [UC-02]

[Insert the sequence diagram for one search, update or report workflow]

[Diagram title / version / date]

7.1 Interaction review

Workflow	Initiating screen	Service method	DAO method	Success result	Error result
[UC-01]	[UI-__]	[method()]	[method()]	[Outcome]	[Message / recovery]
[UC-02]	[UI-__]	[method()]	[method()]	[Outcome]	[Message / recovery]

8 Validation exception handling and data integrity

8.1 Validation design

Input or rule	Validation	Where enforced	User message	Related FR
[Field / rule]	[Required / range / format]	UI / service / database	[Message]	[FR-__]
[Field / rule]	[Duplicate / business rule]	[Layer]	[Message]	[FR-__]
[Field / rule]	[Validation]	[Layer]	[Message]	[FR-__]

8.2 Exception handling

Failure	Detection point	Handling and recovery	Message or log
Invalid input	Swing / service	Prevent operation and keep entered data	Meaningful user message
Database unavailable	DAO / connection	Close resources; allow safe retry or exit	User message + technical log
SQL operation fails	DAO	Rollback if needed; preserve consistency	User message + technical log
Unexpected error	Application boundary	Prevent crash where possible	Generic user message + details in log

8.3 JDBC and data integrity decisions

- Use PreparedStatement for parameterised SQL and to avoid unsafe string concatenation.
- Use transactions for operations that must succeed or fail as one unit.
- Define primary keys, foreign keys, uniqueness and nullability in the database where appropriate.
- Use try-with-resources or an equivalent safe pattern to close JDBC resources.
- Keep database credentials and connection settings outside the source code where feasible.

8.4 Additional project-specific considerations

[Authentication, access control, audit log, backup, concurrency or other considerations relevant to this project.]

9 Design traceability

Use the matrix below to show that every approved functional requirement is supported by a use case, module, class, screen and database element where applicable.

FR	Use case	Module	Class or method	Screen	DB table	Verification
FR-01	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-02	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-03	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-04	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-05	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-06	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-07	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]
FR-08	[UC-__]	[Module]	[Class / method]	[UI-__]	[Table]	[Planned test]

9.1 Important design decisions

Decision ID	Decision	Reason and alternatives considered	Impact
DD-01	[Architecture / library / pattern choice]	[Why this choice fits the project]	[Benefits / limitations]
DD-02	[Database or UI choice]	[Reason and alternatives]	[Impact]
DD-03	[Other important choice]	[Reason and alternatives]	[Impact]

9.2 Design risks

Risk	Likelihood	Impact	Mitigation	Owner
[Design or implementation risk]	L / M / H	L / M / H	[Preventive action]	[Name]
[Risk]	L / M / H	L / M / H	[Mitigation]	[Name]

10 Implementation readiness

Summarise how the team will convert the approved design into the first implementation reviewed in Code Review 1.

10.1 Team responsibility plan

Member	Primary responsibility	Classes or modules	Integration responsibility	Target date
[Team Lead]	[Responsibility]	[Classes / modules]	[Integration / review]	[Date]
[Member]	[Responsibility]	[Classes / modules]	[Integration task]	[Date]
[Member]	[Responsibility]	[Classes / modules]	[Integration task]	[Date]
[Member]	[Responsibility]	[Classes / modules]	[Integration task]	[Date]
[Member]	[Responsibility]	[Classes / modules]	[Integration task]	[Date]

10.2 Design review checklist

[ ] The design implements the approved requirements and scope.

[ ] Use cases, classes, packages, tables and screens use consistent names.

[ ] The design separates Swing UI, business logic and JDBC data access responsibilities.

[ ] Database keys, relationships, validation and error handling are defined.

[ ] Each functional requirement is traceable to design elements and a planned verification.

[ ] The team has recorded current HackerRank Java progress for every member.

10.3 Review outcome

Item	Entry
Decision	[Approved / Approved with changes / Revise and resubmit]
Changes required	[List the key corrections]
Team Lead	[Name and signature]
Reviewer	[Name and signature]
Date	[Date]

Next milestone: Code Review 1 - Basic Java and OOP with emphasis on coding standards. Carry forward the updated HackerRank progress table.

Appendix A HackerRank evidence

Attach one current screenshot per member or provide working profile links. Ensure that the student name or username and Java star level are visible.

A1 HackerRank profile evidence for [Student name]

[Insert current profile screenshot or paste the verified profile URL]

[Diagram title / version / date]

A2 HackerRank profile evidence for [Student name]

[Insert current profile screenshot or paste the verified profile URL]

[Diagram title / version / date]

A3 HackerRank profile evidence for [Student name]

[Insert current profile screenshot or paste the verified profile URL]

[Diagram title / version / date]

A4 HackerRank profile evidence for [Student name]

[Insert current profile screenshot or paste the verified profile URL]

[Diagram title / version / date]

A5 HackerRank profile evidence for [Student name]

[Insert current profile screenshot or paste the verified profile URL]

[Diagram title / version / date]